Bandit - Level 00

Goal

Connect to the Bandit server via SSH as `bandit0`, find the password for `bandit1`.

Connection

```
ssh -p 2220 bandit0@bandit.labs.overthewire.org
# password: bandit0
```

Bandit uses port **2220**, not the default SSH port (22) — common practice for wargame/lab servers to avoid blending in with real SSH traffic on the standard port, and to dodge generic automated scans.

Concepts

SSH (Secure Shell) is a protocol for logging into a remote machine and getting a real shell on it, with the entire session encrypted end to end. Before SSH existed, the standard tool was `telnet` — which sent everything, including passwords, as plain text over the network. Anyone capturing traffic on the path could read it directly. SSH replaced that by wrapping the whole conversation in encryption from the first handshake, so even if the traffic is intercepted, it's unreadable without the right keys.

Under the hood, an SSH connection does two separate jobs:

- **Key exchange & encryption** — client and server agree on a shared secret using asymmetric cryptography, without ever transmitting that secret in the clear. Everything sent afterward is encrypted with it.
- **Authentication** — proving *who* you are, separate from the encryption itself. Bandit uses password authentication (what you typed after the prompt), but in real-world use SSH key pairs (a private key on your machine, a public key on the server) are the stronger, more common method — no password to brute-force, no password to leak.

The command pattern `ssh -p 2220 bandit0@bandit.labs.overthewire.org` breaks down as: `-p 2220` (use this port instead of the default 22), `bandit0` (the username to authenticate as), and the host. This exact pattern — port, user, host — is what you'll use against real servers, homelab VMs, and CTF machines. Bandit isn't a simulation of SSH, it *is* SSH, just pointed at a practice target.

Why port 2220 instead of 22: port 22 is the standard SSH port, which makes it the first thing automated scanners and bots probe on any internet-facing machine. Lab/wargame servers often move SSH to a non-standard port partly to cut down on that background noise, though this is "security through obscurity" — a deterrent, not real protection. The actual security comes from encryption and authentication, not from hiding the port.

The Bandit handoff pattern: each level's home directory contains a `readme` file holding the password for the *next* level. Solve the level's task, read the password, log out, log back in as the next user. This is the core loop for the entire wargame.

Unix permission model, observed live: `/home` on the Bandit server holds 150+ directories, one per level/user across several wargames. As `bandit0`, almost all of them return `Permission denied` when

probed — only `bandit0`'s own home is readable. This is Permissions & Process Management in practice, not theory: ownership and the `r` / `w` / `x` bits actively blocking cross-user access on a real multi-user system.

stdout vs stderr: the wall of `Permission denied` lines from the global `find` search could have been filtered out with `2>/dev/null`, sending only the error stream to the void while keeping the real matches on screen. See Linux - Command Line Reference for the full breakdown of streams and redirection — this was the moment that made the concept concrete instead of abstract.

Solution Summary

Logged in as `bandit0`, ran `cat readme` inside the home directory, retrieved the password for `bandit1`.

Points of Friction

- `cd home` failed — same relative-path habit from the Linux mini-reto: there's no `home` folder *inside* `/home/bandit0`. Recovered with `pwd` + `cd ..`.
- Ran `find -type f -name "readme"` from `/home` to explore globally — returned `Permission denied` for almost every other user's directory, cluttering the output. Could have been cleaned up with `find /home -type f -name "readme" 2>/dev/null` to discard stderr and keep only real matches — not applied in the moment, but the noise itself confirmed the permission model is real and active.
- `cat /bandit18 -name "readme"` — invalid syntax, mixed `find`'s `-name` flag into `cat`, which doesn't have that option.

Key Takeaway

First contact with a real multi-user remote system. The permission boundaries aren't an abstract rule from a note anymore — they're an active wall you hit on every attempt to look outside your own lane. The path forward is always: solve your level → get the next password → reconnect as the next user.

Next

`ssh -p 2220 bandit1@bandit.labs.overthewire.org` using the password retrieved above.

◀ Previous: — · Next ▶ Bandit - Level 01